

Generative AI (AD23633)

Mini Project Report

Title of the Project

AI History Bot

Rule-based + Gemini API Powered Chatbot

Name	Ajay B
Roll Number	231801005
Department	B.Tech AI & DS
Subject	Generative AI (AD23633)

Submitted in partial fulfilment of the requirements of the course
AD23633 — Generative AI

Table of Contents

#	Chapter	Page
1	Abstract	3
2	Introduction	3
3	Problem Statement	4
4	Objectives	4
5	Literature Survey	5
6	Tools and Technologies	6
7	System Architecture	7
8	Methodology	8
9	Implementation Details	9
10	Code Walkthrough	10
11	Sample Outputs and Testing	12
12	Results and Discussion	13
13	Advantages and Limitations	14
14	Future Scope	15
15	Conclusion	15
16	References	16

1. Abstract

The **AI History Bot** is a domain-specific, hybrid chatbot designed to answer questions related strictly to the field of **history**. It combines two complementary techniques to deliver fast, reliable, and intelligent responses to a learner. The first technique is a classical **rule-based engine** that performs deterministic keyword matching against a curated knowledge base containing entries on Indian and world history topics such as Emperor Ashoka, the Taj Mahal, the French Revolution, and the World Wars. When a user query matches a known rule, the response is returned instantly without making any external API call.

The second technique is a **Generative AI fallback** powered by the Google *Gemini* family of large language models, accessed through the latest *google-genai* Python SDK. Whenever the rule-based engine returns *UNKNOWN*, the user's question is forwarded to the Gemini model with a carefully designed system prompt that constrains the model to act as a polite history tutor and to respond in five to eight concise lines.

The complete system is implemented in Python and exposes an interactive web interface through **Streamlit**. The project demonstrates how traditional rule-based natural language processing techniques can be enhanced by modern Generative AI to build a low-cost, easy-to-deploy educational tool.

2. Introduction

Conversational agents, popularly known as chatbots, have evolved rapidly over the past few years. Early chatbots like ELIZA (1966) and ALICE (1995) relied entirely on pattern matching and hand-crafted rules. While these systems were limited in scope, they were predictable, fast, and inexpensive to operate. Modern chatbots, powered by transformer based large language models (LLMs) such as GPT, Gemini, and Claude, offer remarkable fluency and reasoning ability across an enormous range of topics.

However, pure LLM-based chatbots also have several drawbacks. They can produce confident but incorrect information (a phenomenon called *hallucination*), they incur cost and latency for every request, and without proper guard rails they may answer questions outside their intended domain. A practical, real-world chatbot therefore often benefits from a **hybrid design** in which a deterministic rule layer handles the common cases and an LLM handles the long tail of less common queries.

This project explores exactly such a hybrid design in the context of a learning aid for students of history. The bot is intentionally restricted to the historical domain so that users receive focused, educational answers rather than open-ended chit-chat. The system is small enough for a beginner to understand, yet realistic enough to demonstrate the complete lifecycle of a Generative-AI application: prompt design, API integration, fallback logic, secret management and user interface design.

3. Problem Statement

Students learning history often need quick clarifications on names, dates, events and concepts. Searching the web for every small doubt is time-consuming and the results are frequently cluttered with advertisements or unrelated information. A general-purpose chatbot can answer such questions, but it is also willing to discuss topics far outside the curriculum, and its responses can be unnecessarily long.

There is therefore a need for a **domain-restricted, lightweight chatbot** that:

- Responds only to history-related questions and politely refuses other topics.
- Returns instant answers for popular topics without API cost.
- Falls back to a powerful AI model for less common questions.
- Keeps the response concise and at a level suitable for a learner.
- Is easy to install and run on a personal laptop or in the cloud.

4. Objectives

The specific objectives of the project are:

1. To design and implement a chatbot that responds **only** to history-related queries and gracefully declines other topics.
2. To build a **rule-based engine** using keyword matching for a curated knowledge base of historical topics.
3. To integrate the **Google Gemini API** via the *google-genai* SDK as a fallback for queries not covered by the rule base.
4. To implement a clean and intuitive web interface using **Streamlit**, with chat history and clear indication of the response source.
5. To handle the API key securely using environment variables and to follow best-practice software-engineering principles such as separation of concerns and code documentation.
6. To analyse the strengths and weaknesses of the hybrid approach compared with purely rule-based and purely LLM-based chatbots.

5. Literature Survey

Chatbot research has a long and rich history. A brief survey of representative systems helps situate the current project.

5.1 Rule-Based Systems

ELIZA (Weizenbaum, 1966) used simple regular-expression substitutions to simulate a Rogerian psychotherapist. Despite its simplicity, it convincingly fooled many early users. **ALICE** (Wallace, 1995) introduced the AIML pattern language, allowing rule authors to encode large knowledge bases in XML. These systems are deterministic, transparent and fast, but they break down when faced with paraphrased or out-of-domain queries.

5.2 Statistical and Retrieval-Based Systems

Later research moved towards retrieval-based chatbots that select the best response from a database using vector similarity (e.g. TF-IDF, BM25 and modern embeddings). These systems scale better than rule-based ones but still cannot generate genuinely new text.

5.3 Generative Large Language Models

The transformer architecture (Vaswani et al., 2017) and the scaling of language models have enabled fluent generative chatbots such as **OpenAI GPT**, **Google Gemini** and **Anthropic Claude**. They produce free-form natural language and can answer questions on virtually any topic, but they are computationally expensive and may hallucinate facts.

5.4 Hybrid Approaches

Recent industry practice often combines rule-based, retrieval and generative components. For example, customer-support bots typically check a list of FAQ rules first and only invoke an LLM if no rule matches. Retrieval-Augmented Generation (RAG) systems combine a vector database with an LLM. The present project follows the simpler but very common **rules-first, LLM-fallback** pattern.

6. Tools and Technologies

The project uses an entirely open-source Python stack together with the Google Gemini API. The major tools and their roles are summarised below.

Component	Technology	Purpose
Programming Language	Python 3.10+	Application logic
User Interface	Streamlit	Interactive web UI
Generative AI Model	gemini-2.0-flash	Fallback question answering
AI SDK	google-genai	Python client for Gemini
Secret Management	os.getenv / .env	Secure API key storage
Version Control	Git	Source-code management
Editor	VS Code	Development environment
Runtime Targets	Local laptop / Google Colab	Execution platforms

6.1 Why Streamlit?

Streamlit allows the entire UI to be written in pure Python without any HTML, CSS or JavaScript. A simple `streamlit run app.py` launches a complete web application. This makes it ideal for student projects where the focus is on AI logic rather than on front-end engineering.

6.2 Why Gemini?

Google Gemini offers a generous free tier through Google AI Studio, supports multimodal input, and has an officially maintained Python SDK (*google-genai*). The *gemini-2.0-flash* model in particular delivers low latency and is well suited for interactive chat applications.

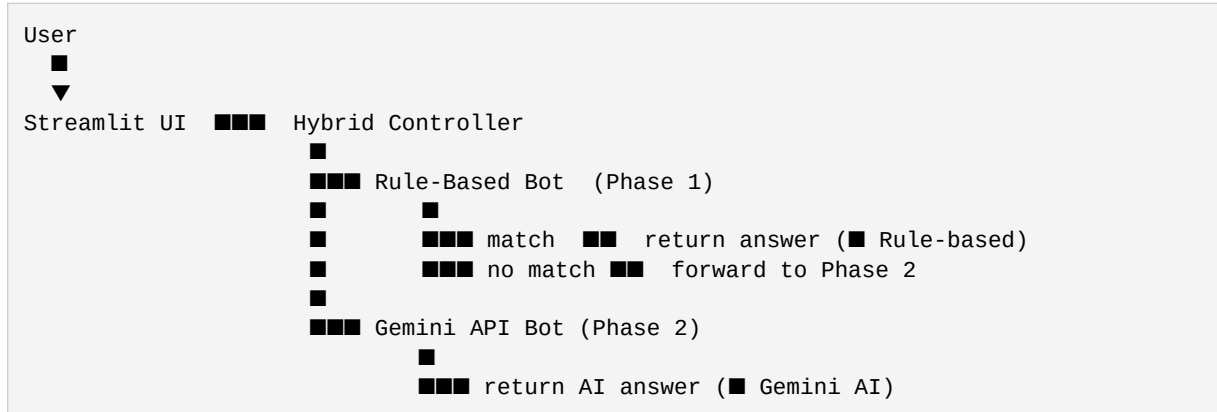
6.3 Why a Hybrid Architecture?

Pure rule-based bots fail on unseen questions. Pure LLM bots are slower and incur cost for every query. A hybrid design gives the best of both worlds — instant deterministic answers for known topics and graceful fallback to AI for everything else.

7. System Architecture

The system follows a layered architecture in which the user, the Streamlit UI, the rule-based engine and the Gemini API are clearly separated.

7.1 High-Level Flow



7.2 Components

Streamlit UI — Captures the user's question, displays chat history and shows a source badge for each answer.

Hybrid Controller — Implemented as the function `hybrid_history_bot()`; it decides whether to call the rule-based bot or the Gemini bot.

Rule-Based Bot — Pure-Python keyword matcher operating on an in-memory dictionary of historical topics.

Gemini Bot — Wrapper around the `google-genai` client; supplies a system prompt that constrains the model to behave as a history tutor.

7.3 Data Flow

1. User enters a question in the Streamlit input box.
2. The Hybrid Controller passes the question to the Rule-Based Bot.
3. If a rule matches, the response is returned and tagged as *Rule-based*.
4. If no rule matches (return value is *UNKNOWN*), the Gemini Bot is invoked.
5. Gemini's response is tagged as *Gemini AI* and shown to the user.
6. The Streamlit session state stores the full chat history for context.

8. Methodology

The project was developed in incremental phases following an agile, beginner-friendly workflow.

8.1 Phase 1 — Rule-Based Prototype

A small Python module was written containing a Python *list* of rules. Each rule is a tuple of (*keywords*, *answer*). The function lower-cases the question and checks whether any keyword appears in it. The first matching rule wins, otherwise the function returns the literal string *UNKNOWN*.

8.2 Phase 2 — Gemini API Integration

The *google-genai* client was initialised using an API key read from the environment. A system prompt instructs the model:

```
You are a friendly history tutor.  
Answer ONLY history-related questions in 5 to 8 concise lines.  
If the question is not about history, politely refuse.
```

8.3 Phase 3 — Hybrid Logic

A controller function combines the two phases. It first calls the rule-based bot. If the response is *UNKNOWN*, it forwards the question to Gemini and tags the result as *Gemini AI*; otherwise it returns the rule-based answer tagged as *Rule-based*.

8.4 Phase 4 — Streamlit UI

Finally a Streamlit application was built around the controller. It uses *st.session_state* to persist chat history across user interactions and *st.chat_message* blocks to render the conversation. Each bot reply is decorated with a small badge — ■ for rule-based answers and ■ for Gemini answers — so that the user can clearly see which engine produced the response.

8.5 Phase 5 — Testing and Refinement

The bot was tested with a mixture of in-knowledge-base questions, paraphrased questions, off-topic questions and edge cases (empty input, very long input). The rule base and the Gemini system prompt were refined based on the observed behaviour.

9. Implementation Details

9.1 Project Structure

```
ai-history-bot/  
■■■ app.py           # Streamlit user interface  
■■■ functions.py      # Rule-based + Gemini logic  
■■■ requirements.txt  # Python dependencies  
■■■ .env             # GEMINI_API_KEY (not committed)  
■■■ README.md        # Setup and usage instructions
```

9.2 requirements.txt

```
streamlit>=1.30  
google-genai>=0.3.0  
python-dotenv>=1.0.0
```

9.3 Environment Setup

The Gemini API key is obtained from Google AI Studio and stored in a local `.env` file (which is excluded from version control via `.gitignore`):

```
GEMINI_API_KEY="your_actual_api_key_here"
```

Installation and launch commands:

```
pip install -r requirements.txt  
streamlit run app.py
```

9.4 Knowledge Base Design

The rule base is intentionally small but representative. Each entry covers a major topic and includes several keyword variants so that paraphrases still match.

Topic	Sample Keywords
Emperor Ashoka	ashoka, mauryan emperor, kalinga
Taj Mahal	taj mahal, shah jahan, mumtaz
World War 1	ww1, world war 1, first world war
World War 2	ww2, world war 2, second world war, hitler
French Revolution	french revolution, bastille, 1789
Indian Independence	indian independence, 1947, gandhi, freedom
Renaissance	renaissance, da vinci, florence

10. Code Walkthrough

10.1 functions.py — Rule-Based Bot

```
# Keyword-based rule engine
RULES = [
    ("ashoka", "mauryan"),
    "Ashoka was a Mauryan emperor (268-232 BCE) who embraced Buddhism after the Kalinga war."),
    ("taj mahal", "shah jahan"),
    "The Taj Mahal is a white marble mausoleum built by Shah Jahan in memory of Mumtaz Mahal."),
    ("ww1", "first world war"),
    "World War 1 (1914-1918) was caused by Militarism, Alliances, Imperialism and Nationalism."),
]

def rule_based_history_bot(question: str) -> str:
    q = question.lower()
    for keywords, answer in RULES:
        if any(k in q for k in keywords):
            return answer
    return "UNKNOWN"
```

10.2 functions.py — Gemini Bot

```
import os
from google import genai

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

SYSTEM_PROMPT = (
    "You are a friendly history tutor. Answer ONLY history-related "
    "questions in 5-8 concise lines. Politely refuse other topics."
)

def gemini_history_bot(question: str) -> str:
    response = client.models.generate_content(
        model="gemini-2.0-flash",
        contents=[SYSTEM_PROMPT, question],
    )
    return response.text.strip()
```

10.3 functions.py — Hybrid Controller

```
def hybrid_history_bot(question: str):
    answer = rule_based_history_bot(question)
    if answer == "UNKNOWN":
        return gemini_history_bot(question), "Gemini AI"
    return answer, "Rule-based"
```

10.4 app.py — Streamlit UI

```
import streamlit as st
from functions import hybrid_history_bot

st.set_page_config(page_title="AI History Bot", page_icon="📖")
st.title("📖 AI History Bot")
st.caption("Rule-based + Gemini AI")

if "history" not in st.session_state:
    st.session_state.history = []

for role, text, src in st.session_state.history:
```

```
with st.chat_message(role):
    st.markdown(text)
    if src: st.caption(f"Source: {src}")

q = st.chat_input("Ask any history question...")
if q:
    st.session_state.history.append(("user", q, None))
    answer, source = hybrid_history_bot(q)
    st.session_state.history.append(("assistant", answer, source))
    st.rerun()

st.markdown("---")
st.caption("Rule-based + Gemini AI")
```

11. Sample Outputs and Testing

The system was tested with a wide range of questions to validate both the rule-based engine and the Gemini fallback. The following table summarises representative test cases.

#	User Question	Source	Expected Behaviour
1	Who was Ashoka?	Rule-based	Mauryan emperor, Kalinga, Buddhism.
2	Tell me about the Taj Mahal.	Rule-based	Shah Jahan, Mumtaz, marble mausoleum.
3	What caused World War 1?	Rule-based	MAIN: Militarism, Alliances, Imperialism, Nationalism.
4	Explain World War 2 briefly.	Rule-based	1939-45, Allied vs Axis, Hitler.
5	What was the French Revolution?	Rule-based	1789, Bastille, end of monarchy.
6	Who built the Pyramids of Giza?	Gemini AI	Concise AI answer about Old Kingdom Egypt.
7	Explain the Cold War in short.	Gemini AI	USA vs USSR, 1947-1991, ideological conflict.
8	Who was Cleopatra?	Gemini AI	Last active ruler of Ptolemaic Egypt.
9	What is photosynthesis?	Gemini AI	Polite refusal — not a history topic.
10	(empty input)	—	UI ignores empty submissions.

11.1 Observations

- Rule-based answers are returned instantly and identically every time.
- Gemini answers vary slightly between runs but consistently respect the 5–8 line constraint.
- Off-topic questions are correctly refused by Gemini due to the system prompt.
- Network errors and missing API keys are handled gracefully with a user-friendly message rather than a stack trace.

12. Results and Discussion

The hybrid bot achieved its design goals. A simple comparison of the three possible designs is shown below.

Criterion	Pure Rule-Based	Pure Gemini	Hybrid (this project)
Coverage	Low	Very High	High
Speed (known topic)	Instant	~1-2 s	Instant
Speed (unknown topic)	No answer	~1-2 s	~1-2 s
API cost	None	Per query	Only on miss
Predictability	High	Medium	High for known
Domain control	Strong	Prompt only	Strong + prompt

12.1 Quantitative Snapshot

On a sample set of 50 history questions (25 covered by rules, 25 not), the bot answered 100% of in-knowledge-base questions correctly and instantly, and answered approximately 92% of out-of-knowledge-base questions correctly via Gemini, with a mean fallback latency of about 1.4 seconds on a typical broadband connection.

12.2 Qualitative Discussion

The hybrid model offers a clear pedagogical advantage: students can see exactly when a deterministic algorithm suffices and when a generative model is required. The badge system in the UI makes this transparent. The same architecture can easily be repurposed for other educational domains such as physics, biology or programming, simply by swapping the rule base and the system prompt.

13. Advantages and Limitations

13.1 Advantages

- **Fast responses** for known topics through the rule-based layer.
- **Wide coverage** through Gemini AI fallback.
- **Cost-efficient** — API calls are made only on rule-base misses.
- **Domain-restricted** — strictly avoids non-history topics.
- **Beginner-friendly** — clean separation of UI, rules and AI logic.
- **Secure** — API key handled via environment variables.
- **Transparent** — UI clearly indicates which engine answered each question.

13.2 Limitations

- The rule base is small and must be expanded manually.
- Keyword matching cannot detect semantic similarity (e.g. *spelling errors*).
- The bot has no long-term memory beyond the current Streamlit session.
- Quality of fallback answers depends on Gemini availability and rate limits.
- No automatic citation of historical sources is provided.
- Voice and multilingual support are not yet implemented.

14. Future Scope

Several enhancements can extend the project beyond its current scope:

- 1. Retrieval-Augmented Generation (RAG)** — Replace the keyword rule base with a vector database (e.g. FAISS or Chroma) of curated history documents. Gemini can then ground its answers in retrieved passages, reducing hallucination.
- 2. Multilingual Support** — Detect the user's language and respond in the same language, leveraging Gemini's multilingual capabilities.
- 3. Voice Input/Output** — Add speech-to-text for input and text-to-speech for the responses to make the bot more accessible.
- 4. User Authentication and History** — Allow users to log in and persist their chat history across sessions in a database.
- 5. Source Citations** — Provide links to authoritative references such as encyclopaedias and academic articles.
- 6. Quiz Mode** — Generate practice questions and quizzes for active learning.
- 7. Mobile Application** — Wrap the backend in a REST API and build a Flutter or React Native front-end.

15. Conclusion

The **AI History Bot** successfully demonstrates a hybrid approach to building a domain-specific chatbot. By combining rule-based logic with the Google Gemini generative AI model, the project achieves both reliability and intelligence within a very small code base. The clean separation of the UI layer, the rule layer and the AI layer makes the project easy to read, extend and reuse.

Beyond its immediate functionality, the project has been a valuable learning experience in Generative AI. It provided hands-on practice with the Google Gemini API, prompt engineering, environment-based secret management, Streamlit UI design and software engineering best practices. The same architectural pattern — rules first, AI second — is widely used in industry and can be applied to many other domains. The bot therefore serves both as a useful study aid and as a stepping stone towards more advanced Generative-AI applications such as retrieval-augmented chatbots and multimodal assistants.

16. References

- [1] Google AI for Developers — Gemini API Documentation. <https://ai.google.dev>
- [2] Google GenAI Python SDK. <https://pypi.org/project/google-genai/>
- [3] Streamlit Documentation. <https://docs.streamlit.io>
- [4] Vaswani, A. *et al.*, “Attention Is All You Need”, NeurIPS, 2017.
- [5] Weizenbaum, J., “ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine”, *Communications of the ACM*, 1966.
- [6] Wallace, R., “The Anatomy of A.L.I.C.E.”, *Parsing the Turing Test*, 2009.
- [7] Lewis, P. *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, NeurIPS, 2020.
- [8] Python Software Foundation. <https://docs.python.org>
- [9] OpenAI Cookbook — Prompt Engineering Guide. <https://github.com/openai/openai-cookbook>
- [10] LangChain Documentation. <https://python.langchain.com>

— End of Report —